树形借用模型

Neven Villani
第一作者
格勒诺布尔大学

Johannes Hostert
第一作者
苏黎世联邦理工大学

Derek Dreyer
作者
马克斯-普朗克
软件系统研究所

Ralf Jung
作者
苏黎世联邦理工大学

Chuigda Whitegive
翻译
Doki Doki λ Club!

Claude
校对
Anthropic

⚠ 注意：本文为早期草稿，内容不完且有措误，且排版质量差。

⚠ Note: this is an early draft. It's known to be incomplet and incorrekt, and it has lots of bad fomatting.

译者前言

本文是文章 Tree borrows 的中文翻译，部分字句有所改动。原文以 CC-BY 4.0 协议发布。术语 *(terminology)* 在正文中第一次出现的地方以仿宋体（中文）或 *Italic (English)* 呈现，如果某个术语难以辨认，则总是会以仿宋体呈现。如遇翻译或排版质量问题，请在 https://github.com/club-doki7/naql/issues 向译者报告。

栈式借用模型 *(Stacked Borrows)* 的概念早在 2020 年就已被提出，而树形借用模型 *(Tree Borrows)* 的思想也在 2025 年得到了形式化，并在 `miri` 中被实际运用。遗憾的是，这些工作在中文 Rust 社区并未得到应有的广泛认知。许多人将 `unsafe` 视作可以为所欲为的免死金牌，在使用 `unsafe` 时忽视别名规则。译者翻译本文的主要动机之一便是纠正这一常见误解。

摘要

Rust 程序设计语言因其基于所有权的类型系统而闻名，这一类型系统为内存安全和无数据竞争提供了强有力的保证。然而，Rust 也提供了非安全 *(unsafe)* 代码这一"逃生舱门"：在 `unsafe` 的领域中并不自动保障安全性，一切全凭程序设计者自觉。这便引出一种矛盾：一方面，编译器希望能尽可能利用类型系统的强力保证——特别是与指针别名相关的保证——以实现强大的过程内优化；另一方面，这些优化易因"行为不良"的非安全代码而失效。因此，要确保优化的正确性，就必须清晰地定义什么样的非安全代码算是"行为不良"。之前的工作——栈式借用模型——为此定义了一系列规则。然而，栈式借用模型排除了几个在实际的 Rust 非安全代码中常见的模式，且并未考虑到 Rust 借用检查器近期引入的一些高级特性。

为解决这些问题，我们提出了树形借用模型。顾名思义，树形借用模型将栈式借用模型的核心替换为树形结构。这就克服了先前所述的限制：我们评估了 30 000 个最常用的 Rust crate，树形借用模型拒绝的测试用例相比栈式借用模型减少了 54%。此外我们还（在 Rocq 中）证明了新模型在保留栈式借用模型的大部分优化的同时，还让一些重要的新优化成为了可能——特别是读-读重排序 *(read-read reordering)*。

1. 简介

类型系统已被证明能以可组合的方式高效地排除整类错误。Rust 正是最近的成功案例，它使用仿射 *(affine)*类型（"基于所有权的"类型）和生存期 *(lifetime,* 区域类型 *region types* 的变体*)* 来静态地确保底层系统应用程序的类型安全、内存安全和无数据竞争，而无需垃圾回收器。

然而，类型系统的好处不只是确保程序不会出错：类型系统还能提高程序的运行速度。事实上，Rust 编译器从一开始就将其类型系统用于优化，而不只是确保安全。特别地，Rust 的类型中编码了强大的别名信息：可变不共享，共享不可变。别名分析是现代优化器的核心支柱，因此 Rust 编译器的开发者希望将无处不在的类型信息用于别名分析也就不足为奇了。考虑这个具体的例子：

```rust
fn write_both(x: &mut i32, y: &mut i32) -> i32 {
    *x = 13;
    *y = 20;
    *x // 返回 13
}
```

例 1

函数 `write_both` 接受两个可变引用作为实参，而根据 Rust 的类型系统规则，这两个引用不可能互为别名。由此可知，该函数必须始终返回 `13`。因此最后一行中从 `*x` 读取数据的操作可被移除，返回值可被硬编码为 `13`，而这些变换不会影响程序的行为。虽说这个例子是人为设计的，但它展示了一类强大的过程内优化：无论内存是在程序中何处分配的，编译器总是可以作这种变换。

1.1. 非安全代码带给别名优化的挑战

不幸的是，Rust 所面临的现实比这复杂得多。Rust 的类型系统所提供的别名保证有时过于强大，使得程序设计者无法实现某些抽象（例如，依赖于共享状态变更的抽象）或是达到预期的性能。因此，程序设计者有时需要使用非安全代码——尤其是原始指针 *(raw pointer)*——来规避类型系统的限制，而原始指针的别名是不受追踪的。Rust 程序设计者会将非安全代码封装在安全的抽象层中以保持类型安全推理的组合性。例如，标准库中的 `Vec` 类型提供了大小可变的数组。`Vec` 类型内部是用非安全代码实现的，但它提供给用户的公共 API 却是安全的。这意味着数百万行代码可以使用 `Vec` 而无需担心类型安全问题，只有实现 `Vec` 的那几千行代码需要额外的审查。

非安全代码的问题在于，用它们可以进行如下操作：

```rust
fn main() {
    let mut x = 42;
    let ptr = &mut x as *mut i32;
    let val = unsafe { write_both(&mut *ptr, &mut *ptr) };
    println!("{val}");
}
```

例 2

以上代码先是创建了指向可变局部变量 `x` 的原始指针 `ptr`，然后用 `unsafe` 代码块，两次将同一个原始指针转型为可变引用，这样就有了两个互为别名的可变引用。因此若不带优化地编译，则这段程序会输出 `20`，但若按照先前所述的方法优化 `write_both`，则程序会输出 `13`。坏！优化不应该改变程序的行为。

Rust 编译器的开发者确是希望支持别名优化的，这就需要某种方法来“排除”上面这样的反例。Rust 程序设计者早已习惯了这一点：使用非安全特性不意味着他们可以为所欲为，编写非安全代码的开发者必须格外注意特殊规则。例如，调用 `get_unchecked` 函数以在不进行边界检查的前提下访问数组时，非安全代码的作者必须确保索引在数组边界之内。不符合这些要求的非安全代码包含未定义行为 *(Undefined Behavior, UB)*，因为由此产生的程序可能会以任意方式出现异常行为。因此，非安全代码的作者有义务确保他们的代码没有未定义行为；而编译器会反过来信任程序设计者，并假定它所编译的代码中没有未定义行为。

要保留例 1 中基于引用的优化，我们需要找到一种方法，使得例 2 中的程序具有未定义行为。但这说来容易做来难。例如，以下完全安全的 Rust 代码片段看似也会导致可变引用别名问题：

```rust
let mut x = 42;
let ref1 = &mut x;
let ref2 = &mut *ref1;
*ref2 = 12;
println!("{ref1}"); // 输出 12
```

例 3

`ref1` 和 `ref2` 都指向 `x`。Rust 若是断然不许可变引用互为别名，又怎能允许这种状况呢？因为这是一种重借用 *(reborrowing)*：`ref2` 派生自 *(derived from)* `ref1`，故编译器允许它们同时存在。然而当 `ref2` 存活时，`ref1` 便不能使用；而当 `ref1` 再被使用时，即是 `ref2` 生存期的终点。若在例 3 的结尾添上一行 `println!("{ref2}")`，就违反了这规则，程序便编译不成了。

因此，摆在我们面前的挑战便是：如何定义“何谓未定义行为”，使得形如例 2 的反例被拒绝，但所有安全 Rust 代码都被接受。事实上，我们还要更进一步，确保足够多“合理”的非安全代码也被接受：必须让程序设计者能以适度的努力编写出正确的非安全代码，非安全 Rust 才真正称得上可用。